

AIM:

- To design and train a Generative Adversarial Network (GAN) for generating realistic handwritten digits using the MNIST dataset.
- To understand the adversarial learning process between Generator and Discriminator.

Algorithm:

1. **Import Libraries:** Use TensorFlow, Keras, NumPy, and Matplotlib.
2. **Define Generator Model:** A neural network to map random noise to fake images.
3. **Define Discriminator Model:** A binary classifier to distinguish real from fake images.
4. **Compile Discriminator:** With binary cross-entropy loss.
5. **Build and Compile GAN:** Freeze discriminator weights during generator training.
6. **Preprocess MNIST Dataset:** Normalize and reshape images.
7. **Train the GAN:**
 - Train discriminator on real and generated images.
 - Train generator via the combined GAN model to fool discriminator.
8. **Visualize Samples:** Plot generated digits during training.

Code:

```
import tensorflow as tf
from tensorflow.keras import layers
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt

# Load EMNIST Digits
```

```

(ds_train, _), ds_info = tfds.load(
    'emnist/digits',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
)

# Preprocessing function
def preprocess(image, label):
    image = tf.cast(image, tf.float32)
    image = (image - 127.5) / 127.5 # Normalize to [-1, 1]
    image = tf.expand_dims(image, -1) # Add channel dimension
    return image

BATCH_SIZE = 256
NOISE_DIM = 100
EPOCHS = 100

train_dataset = ds_train.map(preprocess).shuffle(10000).batch(BATCH_SIZE)

# Generator
def make_generator_model():
    model = tf.keras.Sequential([
        layers.Dense(7 * 7 * 256, use_bias=False,
input_shape=(NOISE_DIM,)),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Reshape((7, 7, 256)),
        layers.Conv2DTranspose(128, 5, strides=1, padding='same',
use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(64, 5, strides=2, padding='same',
use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(1, 5, strides=2, padding='same',
use_bias=False, activation='tanh')
    ])

```

```

        return model

# Discriminator
def make_discriminator_model():
    model = tf.keras.Sequential([
        layers.Conv2D(64, 5, strides=2, padding='same', input_shape=[28,
28, 1]),
        layers.LeakyReLU(),
        layers.Dropout(0.3),
        layers.Conv2D(128, 5, strides=2, padding='same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(1)
    ])
    return model

# Loss functions
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

# Models & optimizers
generator = make_generator_model()
discriminator = make_discriminator_model()
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# Training step
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

```

```

generated_images = generator(noise, training=True)

real_output = discriminator(images, training=True)
fake_output = discriminator(generated_images, training=True)

gen_loss = generator_loss(fake_output)
disc_loss = discriminator_loss(real_output, fake_output)

gradients_of_generator = gen_tape.gradient(gen_loss,
generator.trainable_variables)
gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))

discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))

# Generate and plot images
def generate_and_plot_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i+1)
        plt.imshow((predictions[i, :, :, 0] + 1) / 2.0, cmap='gray')
        plt.axis('off')
    plt.suptitle(f'Epoch {epoch}')
    plt.show()

# Train function
def train(dataset, epochs):
    seed = tf.random.normal([16, NOISE_DIM])
    for epoch in range(1, epochs + 1):
        for image_batch in dataset:
            train_step(image_batch)
        generate_and_plot_images(generator, epoch, seed)

# Start training
train(train_dataset, EPOCHS)

```

Output:

Epoch 100



Result:

A GAN was successfully built and trained to generate realistic handwritten digits. The generator progressively improved to produce visually plausible digit images.

